Two APSP exponent refinements: a complete Lean proof and a conditional bound

Joshua Wharton
QFEX
joshua@qfex.com

20 October 2026

Abstract

We give a complete Lean proof of deterministic integer APSP in $O(n^{2.995560766})$ time in the challenge's exact fixed-width word-RAM model. This retunes the executable parameters of Li Xuanji's verified algorithm, following the numerical certificate of Hrishik Sai Bojnal, and improves the currently listed Lean-verified exponent 2.99556085 by 8.4×10^{-8}. The theorem includes the finite program, correctness, memory and word-size obligations, and its actual execution cost. Separately, we retain a numerical refinement of the leading claimed aligned-cache construction to $O(n^{2.9929479187094})$. Lean proves that faster construction's real numerical certificate and mathematical cost ledger, but its complete machine implementation remains unproved. The two exponents therefore have different formal guarantees.

1 Scope and precise status

The problem is deterministic all-pairs shortest paths in a directed graph with n vertices, polynomially bounded integer weights, and no negative cycles. The challenge requires reachability and distance outputs, with edge-presence and weight matrices as inputs. For each fixed input-magnitude exponent, its word-RAM program must work at every sufficiently large word width proportional to $\log n$ [1].

This note contains two distinct results. Section 2 proves the complete challenge statement at exponent 2.995560766. The subsequent sections concern the faster, conditional exponent 2.9929479187094. The latter is not a fully Lean-verified APSP bound.

The source of the conditional result is the aligned Walsh embedding, polynomial query cache, and composite-incidence reduction in the note by the GitHub author `shreshth-s` [3], building on Alman–Vassilevska Williams [4] and Majdoddin [5]. The pinned source revision is `8a21eaae8159`; its full commit identifier is recorded in `parameters.json`. The challenge classifies its exponent 2.99294792 as *Claimed*: the accompanying Lean development uses a different cost model from the required `EndStatement` model [1].

For that construction, this note proves the numerical feasibility of a new rational witness. The `lean/` development establishes the real-logarithm certificate and its mathematical cost-ledger bound. Its algorithmic corollary uses the cited written lemmas as dependencies. A separate AI-agent review during preparation checked the local identity, Walsh normalization, cache counting, box decomposition, phase costs, certification rule, and reductions to min-plus and APSP; no defect was found in those examined steps. That review and finite checks do not constitute a formal proof of the complete word-RAM program.

2 A complete proof at exponent 2.995560766

Proposition 1 (Complete word-RAM theorem)**.** *The deterministic APSP problem specified by the challenge is solvable in* $O(n^{1497780383/500000000}) = O(n^{2.995560766})$ *time. For every fixed input-magnitude exponent, the program works for all admissible inputs and all word widths above a constant multiple of* $\mathrm{Nat.log2}(n) + 1$*, including the specified empty-input case.*

Proof. The complete formal proof is `verified/Submission.lean` in the accompanying repository. Its final theorem has the exact type

```
theorem Race.bound :
  EndStatement.APSP.SolvedInTime
    ((1497780383 : Rat) / 500000000)
```

The `EndStatement.lean` file is byte-for-byte identical to the challenge definition. This theorem has no undischarged oracle, numerical, correctness, or running-time premise.

The inherited proof is Li Xuanji's complete batched APSP development, with Hrishik Sai Bojnal's earlier numerical refinement [2]. It supplies an actual counting-oracle program in the finite imperative language `Light`, deterministic prime filtering, reductions to min-plus product and APSP, explicit memory layouts and resource bounds, and a proved compiler to the challenge's instructions. We use the following rational parameters, retuning θ and σ in both the code and the proofs:

$$c = 20, \qquad \theta = \frac{5656907}{50000000} = 0.11313814, \qquad \sigma = \frac{3361758}{10^8} = 0.03361758.$$

The new program uses block size $s = 2^{\lfloor \sigma k \rfloor}$, where k is the binary size of n, and query threshold $t = \lceil \theta m \rceil$ at tensor depth m. Changing the executable constants also requires larger constant allowances for parameter computation and intermediate words. Those allowances are updated in the actual execution proofs.

For this inherited ten-term construction, write $\mathcal{H}(x) = -x \ln x - (1-x)\ln(1-x)$ and define

$$\Gamma = \frac{20\mathcal{H}(1/20) - 20\mathcal{H}((1-\theta)/20) - \theta \ln 9}{\ln 4},$$
$$q = \frac{\mathcal{H}(\theta) + \theta \ln 9}{\ln 4}, \qquad B = \frac{19 \ln 3}{\ln 4}.$$

These differ from the aligned-cache quantities in later sections. The new `Batch/Numbers.lean` proves

$$\Gamma > 0.06602549498, \qquad q < 0.43397449436, \qquad B < 15.057143757.$$

Six logarithm enclosures follow from forty terms of the positive atanh series and an explicit geometric remainder. All quantities are Mathlib's actual real logarithms. Rational arithmetic and Lean's kernel check the enclosures; no numerical approximation is assumed.

Let $\eta = 10^{-11}$ be the allowance for the candidate-prime scale. After substituting the conservative bounds above, the three dominant cost exponents are bounded by the following exact terminating decimals:

Cost component	Exponent upper bound
Box preprocessing	2.9955607652902802482502
Queries	2.9955607645685534425436
Pruned encodings	2.99556067980046755757

They follow respectively from $3-2\sigma\Gamma+(1-\Gamma)\eta$, $3-\sigma(1-2q)+q\eta$, and $2+\sigma(2B-1/2)+(B+1/2)\eta$. All are below 2.9955607654. The formal cost proof absorbs constants and logarithmic factors through the sequence

$$2.9955607654 < 2.9955607655 < 2.9955607656 < 2.9955607657 < 2.995560766.$$

The counting-oracle execution theorem `oracle_solves`, its polynomial resource theorem, the filtering host, and the inherited min-plus/APSP and compiler proofs discharge every premise of the final theorem. Thus the exponent bounds actual machine execution, including the required reachability and distance output. □

The decrease from the listed Lean-verified exponent is exactly

$$2.99556085 - 2.995560766 = 0.000000084.$$

This is an asymptotic refinement of the same algorithm, not a claim of practical acceleration or a new underlying APSP algorithm. The faster 2.9929479187094 result below does not follow from this theorem.

Reproduction and attribution. The `verified/` directory contains the full source development, Lean 4.33.1 and Mathlib revision pins, the submission manifest, and `verify.py`. The verifier builds the sources, checks the canonical statement's SHA-256 digest, checks the exact rational theorem type, and rejects transitive axioms beyond `propext`, `Classical.choice`, and `Quot.sound`. Run

```
cd verified
python3 verify.py
```

after installing the pinned dependencies as documented in that directory. The rational-parameter refinement and new certificates are attributed to Joshua Wharton, QFEX. The algorithm, compiler, and inherited proofs retain the credits to Li Xuanji, Hrishik Sai Bojnal, Anthropic, and the original paper's authors. The development was prepared with AI assistance. Local formal verification does not constitute a site review or acceptance by Big O Speedrun.

3 Cost expressions inherited from the construction

Write $\mathcal{H}(x) = -x\ln x - (1-x)\ln(1-x)$ and define

$$C_K = \frac{c\mathcal{H}(1/c)}{\ln 4}, \qquad T = c - 1 + C_K, \tag{1}$$

$$G = \frac{c\mathcal{H}(1/c) - c\mathcal{H}((1-\theta)/c) - \theta\ln 4}{\ln 2}, \tag{2}$$

$$q_1 = \frac{\mathcal{H}(\theta)}{\ln 2}, \qquad q_2 = q_1 + 2\theta, \qquad E = T + q_1. \tag{3}$$

The underlying selected-entry data structure has preprocessing $\widetilde{O}(qND^E + qN^2D^{-G} + qND)$ and a specified-coefficient query cost $\widetilde{O}(qD^{q_1} + D^{q_2})$, where q is the cyclic ring length. The layout condition uses T, whereas cache formation uses E.

Let $R > S \geq 1$ be fixed integers, and put

$$\eta = \frac{b}{R}, \qquad u = S\eta, \qquad v = (R-S)\eta = b - u, \qquad e = b + u.$$

Here n^b is the middle-block scale, n^η is the candidate-prime scale up to logarithmic factors, u is the frozen folded-modulus exponent, and v is the final cyclic-modulus exponent.

At the final cyclic endpoint, the four active runtime savings below exponent three are

$$\boxed{eG-\eta,\quad u-eq_1-\eta,\quad b-eq_2-\eta,\quad 1+\frac{u}{2}-eE-\eta.} \tag{4}$$

The matrix-formation and incidence savings are respectively $1-b-u/2-\eta$ and $b-\eta$. The layout margin is $1+u/2-eT$. The supplied checker also retains the upstream's ordinary-box-encoder checks, even though the aligned cache includes the needed constant coefficients.

4 How the new witness was found

First disregard the arbitrarily small fixed-stage overhead η and balance the four expressions in (4). Equating the first three gives

$$u=e(q_1+G),\qquad b=e(q_2+G),\qquad q_1+q_2+2G=1.$$

The last equality follows from $b+u=e$ and is equivalent to

$$G+q_1+\theta=\frac{1}{2}. \tag{5}$$

Equating the cache saving gives

$$e=\frac{1}{T+(q_1+G)/2}=\frac{1}{T+1/4-\theta/2},\qquad \delta_\infty=\frac{G}{T+(q_1+G)/2}.$$

Thus the search can solve (5) for θ at each c and optimize one scalar objective. A high-precision stationary-point calculation gives approximately

$$\begin{aligned}c_*&=10.665500244065728499,\\ \theta_*&=0.065297823698726309 90,\\ \delta_*&=0.00705208129069065328 1665.\end{aligned}$$

These values guided the rational witness. They do not prove a global optimum or rule out a better algorithm. The certificate below does not depend on the optimizer or on the stationarity equations being solved exactly.

5 The exact rational certificate

Proposition 2. *Take the following decimals as exact rational numbers:*

$$\begin{aligned}c&=10.665500244065729, & \theta&=0.06529782369872631,\\ b&=0.046046352103722105, & \delta&=0.0070520812906,\\ R&=10^{12}, & S&=768979037380.\end{aligned}$$

With (1)–(3) and the exact stage identities above, all 47 numerical inequalities checked in the upstream aligned-cache certificate hold strictly for target saving δ.

Proof. The accompanying `verify_certificate.py` evaluates each expression using rational interval arithmetic. A separate Lean proof establishes the same strict conditions over the actual real logarithm and entropy expressions. The exact inputs are recorded in `parameters.json`. Natural logarithms are enclosed by the explicit series-and-remainder argument in Section 6. The following entries are conservative lower bounds, rounded downward from the computed rational interval endpoints:

Final cost component	Certified saving lower bound
Boxes	0.00705208129064165783200695 57
Residual coefficient queries	0.00705208129062240205919965 89
Stored-box coefficient reads	0.00705208129066091347871730 31
Polynomial cache	0.00705208129105022015581707 74
Ordinary box encodings	0.04246076080665023814655165 91
Matrix formation	0.93624930813844146753893263 54

Each exceeds δ. The smallest runtime gap is greater than $2.24020591996589 \times 10^{-14}$. The final layout margin exceeds 0.03540867951600850580314544 7033.

For completeness, the checker verifies both endpoints of both phases. In the scalar phase, $x \in [0, u]$, the five principal savings are

$$b - x + (b + x)G - \eta, \qquad b - (b + x)q_1 - \eta,$$
$$b - (b + x)q_2 - \eta, \qquad 1 + b - x/2 - (b + x)E - \eta,$$
$$1 - 3x/2 - \eta.$$

In the cyclic phase, $z \in [0, v]$, they are

$$b - u - z + eG - \eta, \qquad b - z - eq_1 - \eta,$$
$$b - eq_2 - \eta, \qquad 1 + b - u/2 - z - eE - \eta,$$
$$1 - 3u/2 - z - \eta.$$

Each is affine in its phase variable. Endpoint verification therefore covers the full interval. The checker also verifies the geometric-tail conditions, increasing-binomial-tail conditions, positivity conditions, tile margins, and exact integer stage identities. There are 19 initial checks and 28 endpoint checks. Full directed lower and upper bounds are recorded in `certificate.json`. □

Corollary 3 (Conditional algorithmic consequence)**.** *Assuming the selected-entry oracle, composite-incidence reduction, min-plus/APSP transfers, and word-RAM cost interpretation in the cited written construction, the stated deterministic integer APSP problem is solvable in* $O(n^{2.9929479187094})$ *time.*

Proof. All runtime terms are bounded by $n^{3-\delta-\varepsilon}$ times fixed powers of $\log n$, for some fixed $\varepsilon > 0$, by the strict certificate. These logarithmic factors are absorbed in n^{ε}. The fixed number of stages, rounding overheads, bit-scaling calls, and APSP squarings are included in that accounting. Since $3 - \delta = 2.9929479187094$, the asserted exponent follows under the stated algorithmic dependencies. □

The improvement over the displayed baseline is exactly

$$2.99294792 - 2.9929479187094 = 0.0000000012906.$$

This is an extremely small asymptotic refinement. The stage count is a fixed 10^{12}; both it and the tiny positive slack imply enormous hidden constants and asymptotic thresholds. No practical speedup is claimed.

6 Why the checker is exact

For rational $1 \le y \le 2$, put $z = (y - 1)/(y + 1)$, so $0 \le z \le 1/3$. The identity

$$\ln y = 2 \sum_{j=0}^{\infty} \frac{z^{2j+1}}{2j + 1}$$

gives the rational enclosure

$$2\sum_{j=0}^{39}\frac{z^{2j+1}}{2j+1} \;\le\; \ln y \;\le\; 2\sum_{j=0}^{39}\frac{z^{2j+1}}{2j+1}+\frac{2z^{81}}{81(1-z^2)}.$$

The upper remainder follows by replacing every omitted denominator by 81 and summing a geometric series. For an arbitrary positive rational x, the checker finds an integer k with $1 \le x/2^k < 2$, then applies $\ln x = \ln(x/2^k) + k\ln 2$, including the correct interval direction when k is negative. All subsequent arithmetic uses Python's `Fraction`; division requires a strictly positive denominator interval. A condition passes only if its lower interval endpoint is strictly greater than its threshold.

Decimal output uses outward rounding and is not used to decide a check. Explicit exceptions, rather than Python assertions, enforce the checks even under optimized execution. The implementation has no third-party dependencies. Running

```
python3 verify_certificate.py --json certificate.json
```

recomputes all 47 inequalities and exact stage identities.

7 Lean certificate for the conditional faster bound

The repository's `lean/` directory contains a Lean 4.33.1 development with Mathlib pinned to revision `0df444a360ea`; the full dependency identifiers are in `lake-manifest.json`. The `LogBounds` module proves seven rational enclosures of actual `Real.log` values using Mathlib's positive atanh-series theorems [6]. These use 40 terms and the valid geometric tail majorant $2z^{81}/(1-z^2)$, which is looser than the Python checker's majorant but sufficient for the chosen endpoints. A proved range-reduction identity then gives enclosures of $\ln 2$, $\mathcal{H}(1/c)$, $\mathcal{H}((1-\theta)/c)$, and $\mathcal{H}(\theta)$ at precision 10^{-24}.

The theorem `ApspBound.Analytic.analyticNumericalCertificate` proves all 47 inequalities at the real cost expressions, with no assumed logarithm bounds. The development also proves the exact stage identities and a common strict reserve of 2×10^{-14} in the inequalities. Affine interpolation establishes the cost conditions throughout both phases, including all fixed $R+1$ stage indices, without enumerating them.

The theorem `ApspBound.Ledger.refinedExponentBigO` proves, for every fixed natural k, that the explicitly defined mathematical ledger `completeCost k` is $O(n^{2.9929479187094})$. It includes a quadratic scan, incidence bookkeeping, six costs at each stage, upward integer rounding, and fixed logarithmic padding. This is a closed theorem about that ledger. A correct word-RAM implementation and domination of its actual execution time by the ledger remain separate proof obligations.

The development uses no admitted proofs or additional project axioms. Its transitive axiom audit allows only the standard `propext`, `Classical.choice`, and `Quot.sound`. The rational certificate uses kernel reduction; no native decision axiom is used. After installing the pinned dependencies, run:

```
cd lean
lake build
lake env lean Audit.lean
```

The repository includes complete setup instructions and an automated workflow for the build and audit. The formalization and research note are attributed to Joshua Wharton, QFEX; the adapted proof arrangement and underlying algorithm retain the upstream attribution stated above.

8 Other avenues examined and verification limits

The search also examined removing channel entries from the polynomial cache, partial channel caching, fixed-profile prefix caching, and larger aligned local identities. The versions with complete cost expressions did not improve the leading exponent. In particular, reconstructing a channel from two ordinary forms reduces preprocessing but increases the residual query exponent; optimizing that tradeoff returned the full cache endpoint. These numerical searches are not impossibility results.

The fact that $I - P$ has bilinear rank two suggests a possible improvement to high-box queries, but the necessary extra stored states lose the ordinary box-tail saving in the straightforward construction. Replacing the query's 4^t factor by 2^t without charging those states would not be justified. A larger improvement needs a new lemma that improves one or more of the four costs in (4).

As corroboration of the inherited proof, three upstream finite checkers were independently rerun: the embedding checker passed 352 original matrix outputs; the cache checker passed 34 arbitrary signed-array queries and 20 encoders; the integration checker passed 530 normalized scalar and cyclic coefficients. These tests do not prove the asymptotic construction. The Lean numerical and cost-ledger proofs for 2.9929479187094 do not discharge that construction's full word-RAM implementation and correctness obligations. This repository records the refinement; it does not constitute a submission to, or verification by, the challenge.

References

[1] Big O Speedrun, *APSP challenge*, accessed 20 October 2026. `https://bigospeedrun.com/apsp`.

[2] Li Xuanji, *Batched deterministic APSP proof*, Big O Speedrun submission 6, with the numerical refinement to 2.99556085 by Hrishik Sai Bojnal (`fringewidth`). The latter development is archived in `bigospeedrun/verifier`, revision `319252b348e2`, directory `submissions/apsp/fringewidth-apsp-299556085`. Pinned complete proof. The underlying Anthropic `formal-math/3sum-apsp` development is pinned there to revision `e1a4e6508154`.

[3] GitHub author `shreshth-s`, *Improving the displayed exponents in arXiv:2610.06783v1*, `thin-matrix-results`, `outputs/exponent-improvement.tex`, revision `8a21eaae8159` (full identifier in the linked source). Pinned source. See especially the aligned Walsh embedding and polynomial-cache lemmas, and equations (A1)–(A13).

[4] Josh Alman and Virginia Vassilevska Williams, *Truly Subquadratic 3SUM and Truly Subcubic APSP via Triangles in Sparse Lopsided Graphs*, arXiv:2610.06783, 2026. `https://arxiv.org/abs/2610.06783`.

[5] Ruhollah Majdoddin, *Thin matrix products up to $D \leq N^{0.172}$ and lossless reductions for 3SUM, Exact Triangle and APSP, with 3SUM over abelian groups and a dichotomy for coordinatewise problems*, arXiv:2610.17420v1, 2026. `https://arxiv.org/abs/2610.17420v1`.

[6] The Mathlib community, *Mathlib 4*, revision `0df444a360ea`, Lean 4.33.1. Real logarithm series and remainder theorems.